

Exp: 09-Write the Python to Implement Travelling Salesman Problem.

Aim:

To write a python program to implement travelling salesman problem.

Algorithm:

Step 1: Initialize the best distance and best route to be infinity and an empty list, respectively.

Step 2: Generate every possible permutation of the cities.

Step 3: For each permutation, calculate the distance of the path by summing the distances between each pair of consecutive cities in the permutation.

Step 4: Add the distance from the last city to the first city.

Step 5: If the calculated distance is shorter than the current best distance, update the best distance and best route.

Step 6: Return the best distance and best route.

Program:

```
import numpy as np
def tsp(cities):
    # Create a distance matrix
    n = len(cities)
    dist_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if i != j:
                dist_matrix[i][j] = np.linalg.norm(cities[i] - cities[j])
    # Initialize variables
    tour = [0]
    unvisited_cities = set(range(1, n))
    total_distance = 0
    # Find the nearest unvisited city and add it to the tour
    while unvisited_cities:
```

```

current_city = tour[-1]
nearest_city = min(unvisited_cities, key=lambda city: dist_matrix[current_city][city])
tour.append(nearest_city)
unvisited_cities.remove(nearest_city)
total_distance += dist_matrix[current_city][nearest_city]
# Complete the tour by returning to the starting city
tour.append(0)
total_distance += dist_matrix[tour[-2]][0]
return tour, total_distance

```

Output:

```

Tour:  [0, 1, 2, 3, 0]
Total Distance: 16.012869126098966

```

Result:

The Travelling Salesman Problem was successfully solved using the Nearest Neighbor Heuristic approach.